

Introduction to PostgreSQL Database for Full-stack Application

Understanding Data Management

- Almost every piece of software ever written works with data.
- The most common way to manage data is to use a database.
- Database technology is particularly useful for business software, which typically requires sets of related information.
- This type of information is best described using a *relational model* which is the philosophy that underlies all modern database products, including SQL Server and Oracle.

Examples of Database driven Websites

- E-commerce sites (like Amazon) use detailed databases to store product catalogs. They also track orders, customers, shipment records, and inventory information in a huge arrangement of related tables.
- Search engines (like Google) use databases to store indexes of page URLs, links, and keywords.
- Knowledge bases (like Microsoft Support) use less structured databases that store vast quantities of information or links to various documents and resources.
- Media sites (like The New York Times) store their articles in databases.

Database Access in the Web World

- Web applications raise two new considerations: problems of scale and problems of state.
- **Scale:**
 - The problems that can result from the massively multiuser nature of the Web.
 - This means it can't be casual about using server memory or limited resources such as database connections
 - Database concurrency issues can cause significant problems such as failed updates and inconsistent data.

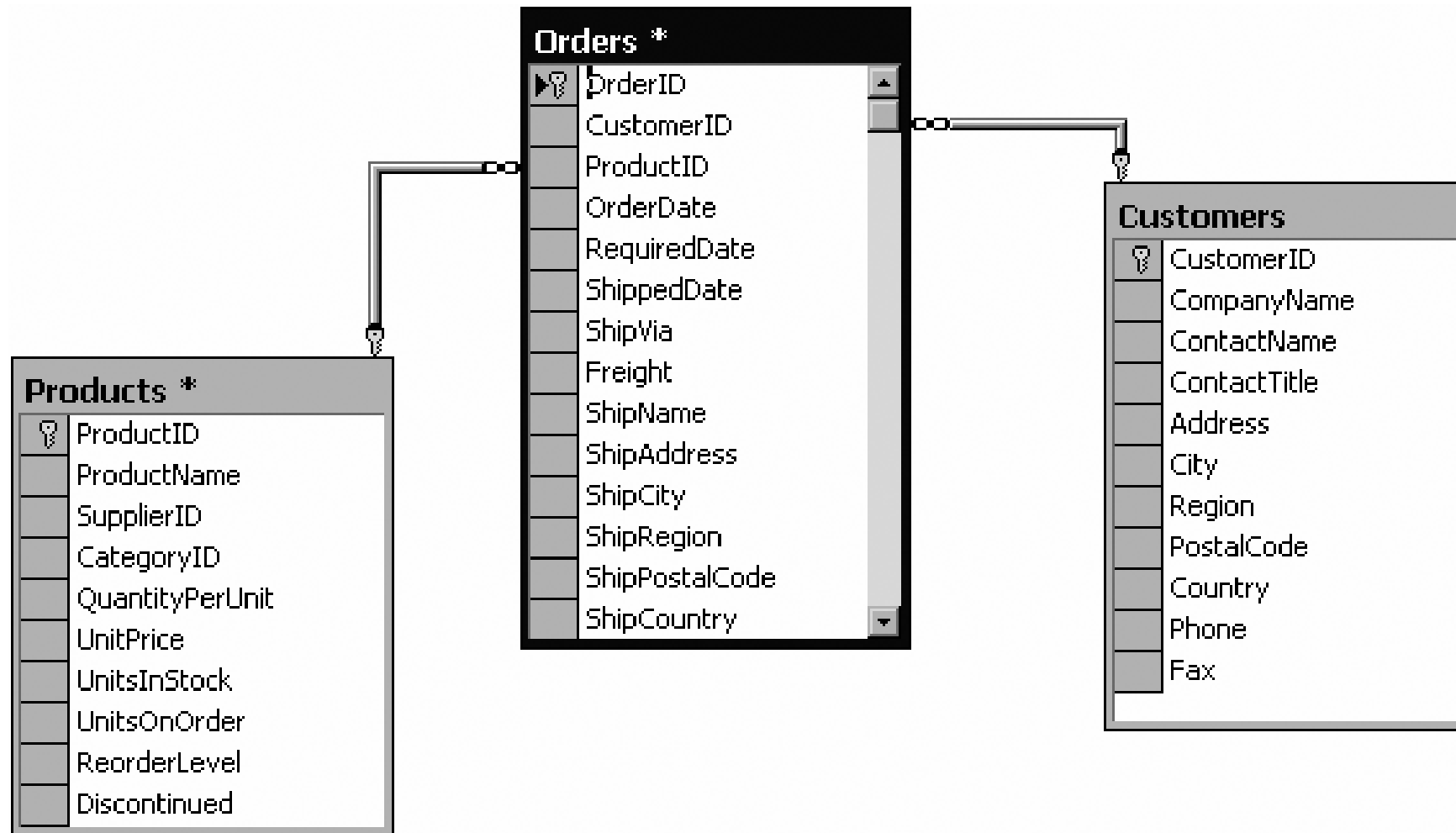
Database Access in the Web World

- **State:**

- Problems of *state* are problems that can result from the disconnected nature of the Internet.
- As you already know, HTTP is a stateless protocol. When a user requests a page in an ASP.NET application, the web server processes the code, returns the rendered HTML, and closes the connection immediately.
- The typical approach is to connect to a database, read information, display it, and then close the database connection.
- This approach runs into difficulties if you want the user to be able to modify the retrieved information.
- In this scenario, the application requires a certain amount of intelligence in order to be able to identify the original record, build a SQL statement to select it, and update it with the new values.

Data Model

- Flat file
- Relational
- Key-Value
- Document
- Columnar
- Graph
- Search
- Time-Series
- In-memory
- Object



Relational databases

- Oracle
- SQL Server
- MySQL
- MariaSQL
- Postgres

Why Postgres?

- Jack-of-all-trades (much like Python on the programming side)
- Open source (easier licensing)
- Gaining popularity
- Performance

SQL Basics

- A Select statement retrieves records.
- An Update statement modifies existing records.
- An Insert statement adds a new record.
- A Delete statement deletes existing records.

The Select Statement

- To retrieve one or more rows of data, you use a Select statement. A basic Select statement has the following structure:

```
SELECT [columns]  
FROM [tables]  
WHERE [search_condition]  
ORDER BY [order_expression ASC | DESC]
```

- A Sample Select Statement

```
SELECT * FROM Authors
```

Improving the Select Statement

- Here's another example that retrieves a list of author names:

```
SELECT au_lname, au_fname  
FROM Authors  
WHERE State='CA'  
ORDER BY au_lname ASC
```

The Where Clause

```
SELECT *  
FROM Sales  
WHERE ord_date < '2000/01/01' AND ord_date > '1987/01/01'
```

String Matching with the Like Operator

```
SELECT * FROM Stores WHERE stor_name LIKE 'B%'
```

```
SELECT * FROM Stores WHERE stor_name LIKE '%B'
```

```
SELECT * FROM Stores WHERE stor_name LIKE '%book%'
```

Aggregate Queries

Table 15-1. *SQL Aggregate Functions*

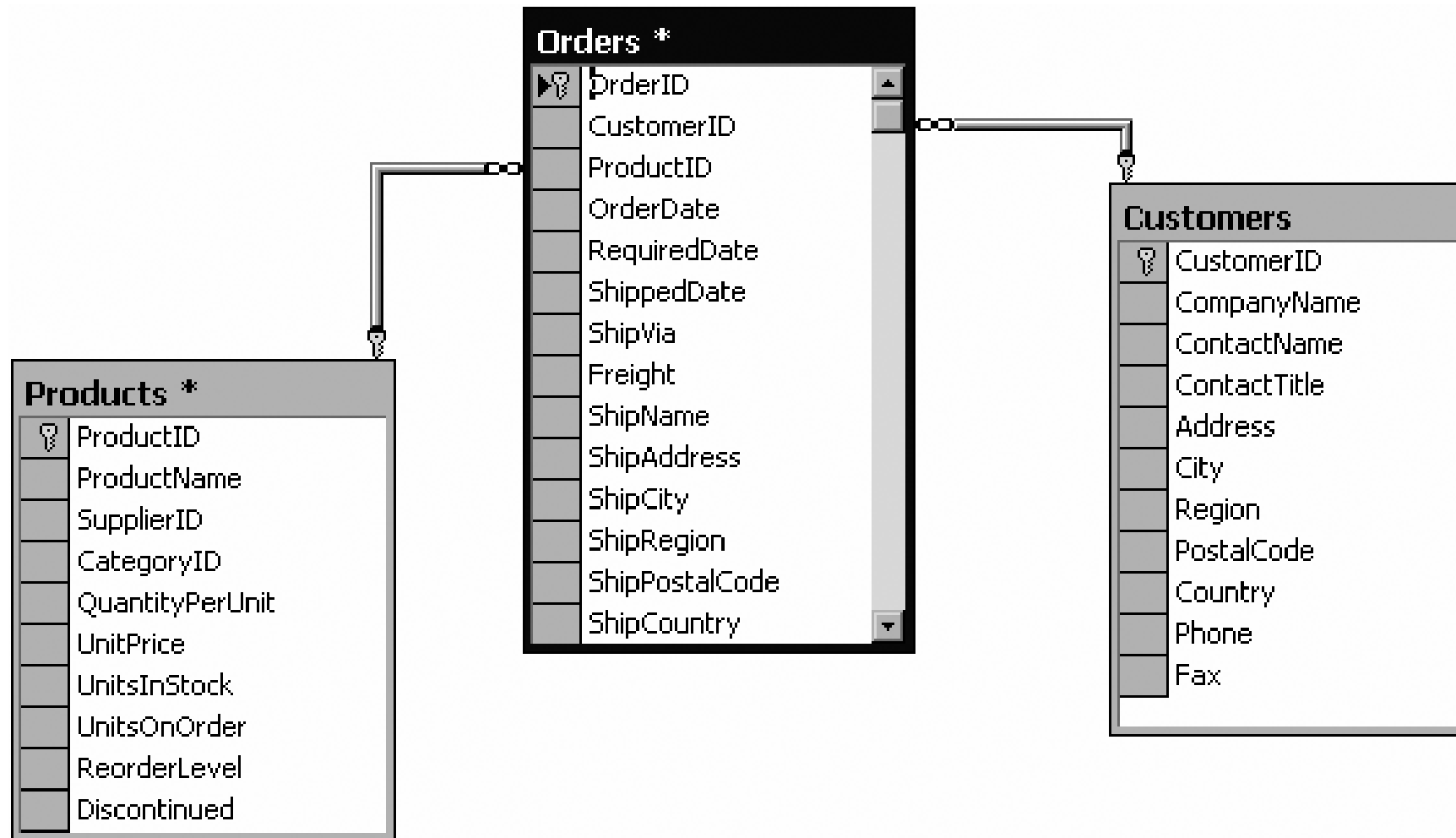
Function	Description
Avg(fieldname)	Calculates the average of all values in a given numeric field
Sum(fieldname)	Calculates the sum of all values in a given numeric field
Min(fieldname) and Max(fieldname)	Finds the minimum or maximum value in a number field
Count(*)	Returns the number of rows in the result set
Count(DISTINCT fieldname)	Returns the number of unique (and non-null) rows in the result set for the specified field

Aggregate Queries Example

- For example, here's a query that returns a single value—the number of records in the Authors table:

```
SELECT COUNT(*) FROM Authors
```

Create these 3 tables:



The SQL Update Statement

- The SQL Update statement selects all the records that match a specified search expression and then modifies them all according to an update expression.
- `UPDATE [table] SET [update_expression] WHERE [search_condition]`
- Example:
- `UPDATE Authors SET phone='408 496-2222' WHERE au_id='172-32-1176'`

The SQL Insert Statement

- The SQL Insert statement adds a new record to a table with the information you specify. It takes the following form:

```
INSERT INTO [table] ([column_list]) VALUES ([value_list])
```

```
INSERT INTO Authors (au_id, au_lname, au_fname, zip, contract)  
VALUES ('998-72-3566', 'Khan', 'John', 84152, 0)
```

The SQL Delete Statement

- The Delete statement is even easier to use. It specifies criteria for one or more rows that you want to remove. Be careful: once you delete a row, it's gone for good!

```
DELETE FROM [table] WHERE [search_condition]
```

```
DELETE FROM Authors WHERE au_id='172-32-1176'
```

Tools used this week

- **PostgreSQL**: object-relational database system
- **pgAdmin4**: GUI for PostgreSQL (comes with installer)
- **Flask**: Python web app framework
- **Psycopg2**: PostgreSQL database adapter
 - Psycopg 3 is a modern implementation of a PostgreSQL adapter for Python.
 - `pip install psycopg` ?
- **SQLAlchemy**: DB communication layer.
 - It translates classes to tables, converts function calls to SQL statements.
 - So we don't have to write SQL statements in code and worry about the syntax
- **Pipenv** (optional): Creates and manages a virtualenv (so no global install)
- Download Postgres - <https://www.postgresql.org/>

What is PostgreSQL?

- PostgreSQL is object-relational database system
- Similar to the MySQL Database management system, PostgreSQL is another type of RDBMS used for accessing, storing, and handling the data in the form of database tables.
- PostgreSQL also uses SQL- Structured Query Language to access and handle databases and also perform various Tasks in PostgreSQL
- Download Postgres - <https://www.postgresql.org/>

What is Object Relational Mapper (ORM)

- Object-Relational Mapping (ORM) is a technique that lets you query and manipulate data from a database using an object-oriented paradigm.
- **ORM** - library that implements the Object-Relational Mapping technique
- With flask, *Object Relational Mappers* (ORM's) are employed to allow app to interact with a relational database.
- An **Object Relational Mapper** is a **framework** that, in our case, will **allow us to interact with a SQL database using python instead of explicit SQL queries**.

ORM & SQLAlchemy

- An Object Relational Mapper is a framework that, in our case, will allow us to interact with a PostgreSQL database using python instead of explicit SQL queries.
- The name of the ORM we are using is [SQLAlchemy](https://www.sqlalchemy.org/) and can be downloaded as follows:
 - **pip install flask_sqlalchemy**
 - <https://www.sqlalchemy.org/>
 - Psycopg 3 is a modern implementation of a PostgreSQL adapter for Python.
 - pip install psycopg

Flask Marshmallow

- Flask-Marshmallow is a thin integration layer for Flask (a Python web framework) and Marshmallow (an object serialization/deserialization library) that adds additional features to marshmallow, including URL and Hyperlinks fields for HATEOAS-ready APIs.
- It also (optionally) integrates with Flask-SQLAlchemy.
 - <https://flask-marshmallow.readthedocs.io/en/latest/>
- Download
 - **pip install flask-marshmallow**
- To enable SQLAlchemy integration, you need both Flask-SQLAlchemy and marshmallow-sqlalchemy are installed.
 - **pip install -U flask-sqlalchemy marshmallow-sqlalchemy**

OPTIONAL: Create virtual environment for app

- Create a virtual environment in which Flask is installed. Using a virtual environment avoids installing Flask into a global Python environment and gives you exact control over the libraries used in an application.
 - A virtual environment also makes it easy to Create a requirements.txt file for the environment.
- **Steps:** <https://code.visualstudio.com/docs/python/tutorial-flask>
 1. Create a project folder such as “customers_api”
 2. In that folder, run command below to create & activate virtual environment
 - Windows:
 - “python -m venv .venv”
 - “.venv\scripts\activate”
 - Mac
 - python3 -m venv .venv
 - source .venv/bin/activate

Product API with SQLite

- Install Dependencies

- Pip install -r requirements.txt

- Create Database

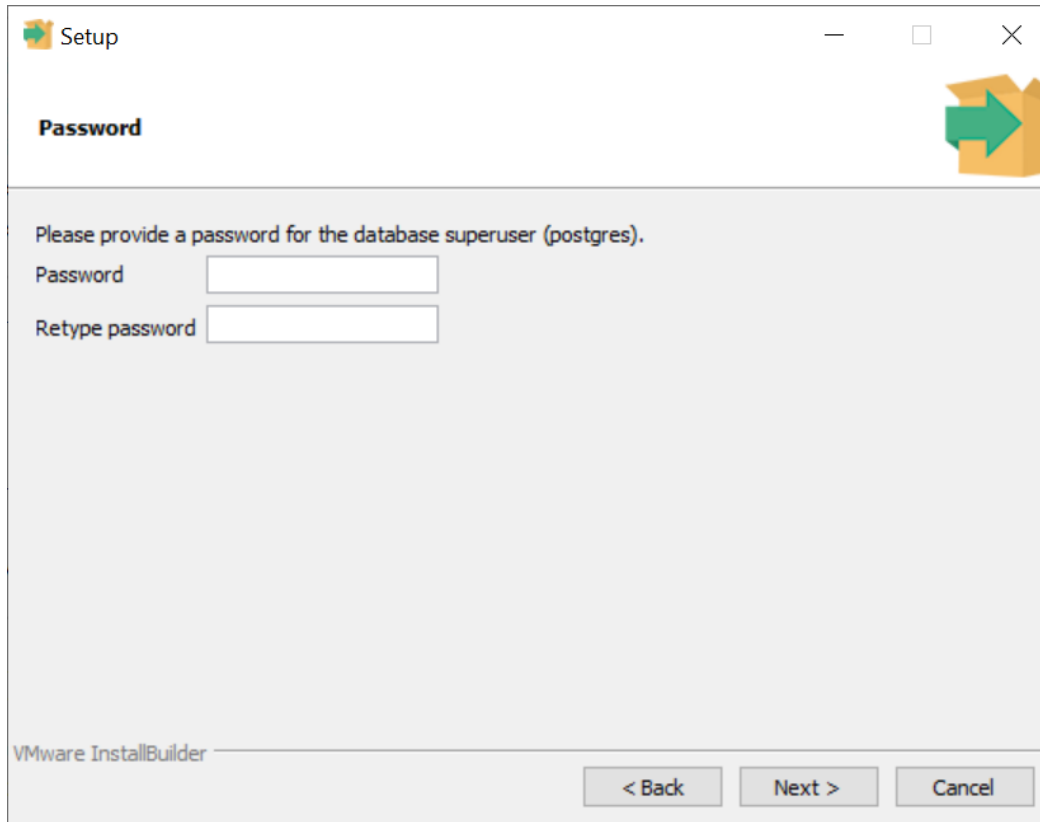
- Open Anaconda prompt
 - \$ python
 - >> from <python file name> import db
 - Example: from product_api_sqlite import db
 - >> db.create_all()
 - >> exit()

- Run Server (<http://localhost:5000>)

- python <python file name>.py
 - Example: python product_api_sqlite.py

PostgreSQL Local Access

- Default Port: 5432
- Default User: postgres



The screenshot shows a window titled "Setup" with a standard Windows title bar (minimize, maximize, close buttons). The window has a light gray background. At the top left, there is a small icon of a green square with a white plus sign. Below the title bar, the word "Password" is displayed in bold. To the right of the title bar, there is a large green arrow pointing right, overlaid on a yellow box. Below this, the text "Please provide a password for the database superuser (postgres)." is shown. Underneath, there are two input fields: "Password" and "Retype password". At the bottom left, the text "VMware InstallBuilder" is visible. At the bottom right, there are three buttons: "< Back", "Next >", and "Cancel".

Setup

Password

Please provide a password for the database superuser (postgres).

Password

Retype password

VMware InstallBuilder

< Back Next > Cancel

What does it mean to master full stack skills?

Mastering full stack skills involves gaining proficiency in a diverse set of technologies.

- On the frontend, developers work with languages such as HTML, CSS, and JavaScript to create a seamless and visually appealing user experience.
- Simultaneously, on the backend, they delve into server-side languages like Node.js, Python, or Ruby, ensuring that the application runs smoothly and efficiently.
- This broad skill set allows for greater collaboration between different development teams and facilitates a more streamlined and efficient workflow.
- Companies are increasingly seeking professionals who can handle the entire web development process, from concept to deployment.